Doramagic 产品定义手册

> 版本：v2.0 完整版 | 日期：2026-03-11 | 状态：讨论中

一、工作方法

讨论方式：第一性原理沟通，一次讨论一个问题，追问本质。

二、核心理念

不是「帮你做」，而是「帮抄作业」 —— 快速动起来。

Doramagic 的核心能力是**提取灵魂**： - 洞察开源项目做了什么 - 为什么要做 - 终局要解决什么问题

学习引擎从人类变成 AI 大模型 —— 用 AI 去学习和提炼，而不是靠人肉抄作业。

三、产品定位

定位迭代

版本	描述
v1.0 产品手册	解决「AI 对开源项目一知半解」的问题，双半魂模型
v2.0（本次讨论）	**超级 copy cat** — 站在巨人肩膀上快速抄作业

竞品差异

产品	定位
Cursor/Copilot	建筑师 - 从 0 构建
Doramagic	抄作业专家 - 复制 + 调整

四、范式转变（核心洞察）

洞察：「好」vs「适合」

之前	AI 时代
用户适应工具	工具适应用户
忍受不喜欢的功能	AI 动态调整

范式对比

旧范式	新范式
学习成本：高	使用成本：零
用户适应工具	工具适应用户
教用户开发	直接给成品

五、三个核心挑战

挑战	优先级	说明
噪音过滤	⭐⭐⭐ 最高	准确识别知识和经验，是产品成立的前提

挑战	优先级	说明
关联推理	⭐⭐ 很高	用户表达缺失，如何准确映射到开源项目
时效性	⭐ 次要	知识是沉淀的，不需要实时性

六、触发词设计

设计原则

- 简洁：「Dora」
- 语言变体存在差异（中文「哆啦A梦」、日文「ドラえもん」、英文「Doraemon」）

Doraemon 风格

- 语气俏皮、友好
- 从「口袋」掏工具的感觉
- 帮助用户解决问题

示例交互

用户: Dora 我想管 Trello

Doramagic: 了解啦！让我从口袋里找找看～🎁

你是想用 Trello 做什么呢?

A) 日常管理任务 - 创建卡片、移到不同列表、标记完成 B) 同步进度 - 查看各 Board 卡片、追踪任务状态 C) 团队协作 - 留评论、@人、分享卡片

七、用户交互设计

自适应交互深度

核心洞察：**不是所有用户都需要深度交互**

用户类型	需求清晰度	需要的交互
能手型	说得很清楚	0 轮，直接进入选择
普通型	大概知道	1-2 轮选择题
新手型	说不清楚	3-5 轮追问

置信度机制

- 阈值：70%
- 置信度 > 70% → 直接呈现能力积木
- 置信度 < 70% → 启动选择题（最多 3 轮）
- 轮数达到上限 → 强制进入下一阶段，提供保守方案

关键设计原则

原则	含义
选择题优先	不让用户自己组织语言
最多 3 轮	超过后主动提供保守方案
提前终止	用户一旦选了就立刻推进
复述确认	「你是说 xxx 对吧？」形成闭环

八、第四阶段：呈现能力积木

四种积木类型

类型	定义
触发积木	用户的初始需求表达

类型	定义
能力积木	开源项目的核心能力
资源积木	API、工具链、信息源
依赖积木	配置、认证（对用户透明）

资源来源

外部资源（可提取） - GitHub 开源项目（代码 + Issues + Discussions） - OpenClaw 现有 Skills

用户上下文（实时整合） - 可用模型（MiniMax, Claude...） - 记忆文件 - 内置工具 - 已安装 Skills

四层提取体系

层次	内容
代码层	设计意图、架构决策
社区层	踩坑经验、workaround
资源层	API、工具链、信息源
上下文层	记忆、偏好、模型

「四足够」设计原则

原则	实现
足够专业	分析用户 OpenClaw 上下文（模型、记忆、工具）
足够智能	子代理并行工作，一次性呈现结果
足够适用	开箱即用 + 配置引导 + 增强建议

原则	实现
足够透明	实时进度条 + 阶段可见 + 预计时间

处理环节

环节	做什么	产出
1. 上下文分析	读取用户 OpenClaw 资源	可用模型、记忆、工具、已安装 Skill
2. 意图理解	理解用户想要什么	意图 + 场景 + 置信度
3. 资源检索	从积木库检索相关积木	候选积木列表
4. 上下文匹配	结合用户上下文个性化推荐	匹配度 + 推荐理由
5. 依赖检查	检查积木所需的依赖配置	缺失配置 + 解决建议
6. 呈现生成	生成最终的用户界面	积木列表 + 增强建议

九、技术架构

技术选型历程

阶段	方案	问题
最初	LangChain + Qdrant + BGE + Mem0 + Neo4j	太重，不是 Skill
最终	纯 LLM + OpenClaw Skills + 记忆文件	轻量、纯 Skill

最终轻量架构

```
用户 → Doramagic Skill ↓ ┌──────────────────────────────────────┐ │ 纯 LLM 能力 │ │ • 意图理解（Prompt） │ │ • Skill 规划（LLM） │ │ • 上下文读取（记忆文件） │ │ • 结果组装生成 │
```

```
└──────────────────────────────────────┘ ↓ Skill 调用层（Skills + LLM + 记忆）
```

零外部依赖

资源	来源
LLM	OpenClaw 已配置
Skills	OpenClaw 已有
知识存储	记忆文件
灵魂提取	已有 soul-extractor Skill

关键技术组件

组件	功能	技术方案
意图识别	理解用户想要什么	LLM + Prompt Engineering
置信度评分	判断需求是否清晰	四维度加权模型
选择题生成	生成合理的选项	LLM 生成 + 知识库检索
对话状态	管理多轮对话状态	内存状态机
灵魂提取	从开源项目提取知识	soul-extractor Skill（已存在）

置信度评分维度

维度	权重
实体完整度	35%
意图匹配度	25%

维度	权重
模糊词检测	20%
上下文连贯性	20%

十、长期记忆系统

核心方向

记住用户的「适合」—— 动态调整，而非固定选项。

记忆维度

维度	说明
喜欢什么方式	用户偏好什么交互风格
跳过什么建议	用户不关注哪些信息
选择什么配置	用户最终决策模式
关注什么信息	用户注意力焦点

例子说明

用户 A - 说「帮我做个 Trello 管理」 - Doramagic 给出 4 个选项：A/B/C/D - 用户 A 选了「简洁模式」 - → 长期记忆：A 选择了简洁模式，不想要复杂配置 - 下次用户 A 说「帮我做个日历管理」 - → Doramagic 直接默认给简洁模式，跳过复杂选项

用户 B - 每次都选 D（高级自定义） - → 长期记忆：B 喜欢深度定制 - 下次给更多高级选项

十一、用户场景

场景 1：告警规则不生效

- **用户问题**：「为什么我的告警规则不生效？」
- **通用 LLM**：告诉你「在 alertmanager.yml 里加 group_by 参数」
- **Doramagic**：
- 代码层面：为什么要这么配
- 社区层面：GitHub Issue 里的 bug report、别人的踩坑经验、最新的 work-around

场景 2：Vibe Coding 新手

- 用 Cursor/Claude Code 写代码
- 遇到问题和实际跑起来的结果不一样
- 需要快速定位是「配置问题」还是「版本 bug」还是「踩过的坑」

十二、待讨论

- [ ] 长期记忆的技术实现方案
- [ ] Doramagic 具体的产品形态
- [ ] 商业模式

十三、相关文档

- soul-extractor Skill（已存在）
- 技术方案研究报告（已生成）
- 能力积木设计文档（已生成）

本版本基于 2026-03-11 完整讨论整理 工作方法 → 核心理念 → 产品定位 → 范式转变 → 核心挑战 → 触发词 → 用户交互 → 能力积木 → 技术架构 → 长期记忆 → 用户场景